rococo.gui.retained: Layout Problems

Source: github.com/theabstraction/rococo, branch master, folder source/rococo/rococo.gui.retained (mainly rococo.gr.panel.cpp). Found by reading the code on 2 November 2026; nothing was compiled or run, so treat these as likely issues to confirm.

Summary of the layout pass

1. LayoutBeforeFit hooks run top-down (widgets set constant sizes, e.g. labels fit their text).
2. ShrinkToFitRecursive runs bottom-up: sums child sizes along the layout direction, takes the maximum across it; FitChildren panels take that size plus padding.
3. LayoutBeforeExpand hooks run top-down.
4. ExpandToParentRecursive runs top-down: free space along the direction is split evenly among ExpandToParent children; across the direction, expanding children fill the parent minus padding.
5. LayoutAfterExpand hooks run top-down.
6. SetAbsRectRecursive computes screen rects: parent rect + parentOffset + parent left/top padding. Flowing parents overwrite each child's parentOffset; only direction None keeps hand-set offsets.

Layout runs every frame from RenderAllFrames; there is no dirty flag, and InvalidateLayout is never called.

Problems

1. BottomToTop is never positioned

Severity: High

Problem: The BottomToTop case in SetAbsRectRecursive (panel.cpp around line 699) is empty, so children of a BottomToTop panel never receive absolute rects, although the expand pass does handle that direction.

Suggested fix: Implement BottomToTop positioning mirroring TopToBottom: start at Height minus bottom padding and step upwards by child height plus childPadding.

2. RightToLeft offsets are wrong

Severity: High

Problem: Positioning starts at Width minus Padding().right and subtracts childPadding before placing the first child (around lines 667 to 679). The child rect then also adds the parent's

Padding().left. The first child sits one gap too far left and every child is shifted by the left padding.

Suggested fix: Subtract childPadding only between children, and compute offsets so that adding the left padding later lands correctly (or exclude the left padding for this direction).

3. Fit pass ignores childPadding

Severity: High

Problem: ShrinkToFitRecursive does not include childPadding in its sum (and does nothing for direction None), while expand and positioning do. A FitChildren list with a non-zero gap is too small and its last child overflows. The viewport's own domain calculation (viewport.cpp around line 56) does include gaps, so the two disagree.

Suggested fix: Add (visibleChildCount - 1) x childPadding along the layout direction in the fit pass, skipping collapsed children consistently with the other passes.

4. Fit runs before expand

Severity: Medium

Problem: A FitChildren parent containing ExpandToParent children measures them using sizes left over from the previous frame. Results depend on history, and nested fit/expand combinations can take several frames to settle or may oscillate.

Suggested fix: Either forbid or warn about fit-over-expand on the same axis, or have expanding children contribute their minimum or fixed content size during the fit pass.

5. Hand-set offsets silently overwritten

Severity: Medium

Problem: The default layoutDirection is LeftToRight (panel.cpp around line 512), so SetParentOffset has no effect unless the parent is set to None. The splitter sets SetParentOffset({offset,0}) on its second child while its own panel appears to flow LeftToRight (splitter.cpp around lines 34 and 77), so the offset is probably overwritten (uncertain).

Suggested fix: Default to None, or assert/warn in debug builds when SetParentOffset is called on a child of a flowing parent. Check the splitter.

6. Expanding children collapse to zero silently

Severity: Low

Problem: When free space is zero or negative, ExpandToParent children get size 0 with no warning.

Suggested fix: Log or flag in debug builds; consider minimum sizes.

7. Rounding remainder dropped

Severity: Low

Problem: freeSpace / n integer division discards the remainder, leaving a gap of up to n-1 pixels at the end.

Suggested fix: Give the remainder to the last child, or spread one pixel each across the first children.

8. Percentage height not handled in vertical expand

Severity: Medium

Problem: The horizontal expand handles ConstantPercentage widths (around line 735) but the vertical expand just uses Span().y (around line 826).

Suggested fix: Mirror the horizontal percentage handling in the vertical expand.

9. Percentage padding based on a changing size

Severity: Low

Problem: Percentage padding is computed from the panel's own span, which changes during the pass, so padding can differ between passes in the same frame.

Suggested fix: Resolve percentages once per pass against a fixed reference size, preferably the parent's.

10. Resize() does not change the sizing rule

Severity: Low

Problem: Resize() sets the span but leaves the sizing rule in place, so an ExpandToParent or FitChildren rule overrides it on the next frame.

Suggested fix: Have Resize() set Constant rules, or document that it is temporary.

Missing features worth noting

- No minimum or maximum sizes.
- No anchors or docking, despite the GRAnchorPadding name.
- No cross-axis alignment of children; GRAlignmentFlags only affect text and images inside a widget.
- No layout invalidation; layout is recomputed every frame.

Debugging suggestions

- Debug overlay: draw each panel's rect, padding and gaps, labelled with Desc() and both sizing rules.
- Extend IGRPanelWatcher with fit and expand callbacks that report size before and after, the rule and the free space, so one panel's path from the root can be traced.
- Layout-check mode: warn on zero-sized expanding children, SetParentOffset under a flowing parent, mixed fit and expand on one axis, and rects that differ from the previous frame.
- Snapshot tests of rects for each direction and sizing rule combination.